

File Directory :

blockchain-prediction-market/	# Root folder
├─ contracts/	# Smart contracts
│ └─ PredictionMarket.sol	# Main Solidity contract
│ └─ Migrations.sol	# Truffle migrations (auto-generated)
├─ migrations/	# Deployment scripts
│ └─ 1_deploy_contracts.js	# Deploys PredictionMarket
│ └─ 2_deploy_mock_oracle.js	# (Optional) Oracle integration
├─ test/	# Unit tests
│ └─ predictionMarket.test.js	# Test cases for the contract
├─ frontend/	# Web-based frontend (React.js)
│ └─ public/	# Static files (HTML, images)
│ └─ index.html	# Main HTML file
│ └─ src/	# React components
│ └─ components/	# Reusable UI components
│ └─ CreatePrediction.js	
│ └─ PredictionList.js	
│ └─ BetForm.js	
│ └─ App.js	# Main React component
│ └─ config.js	# Contract ABI and address
│ └─ index.js	# React entry point
│ └─ styles.css	# Custom CSS
├─ package.json	# Frontend dependencies
└─ README.md	# Frontend docs
├─ truffle-config.js	# Truffle network settings
└─ package.json	# Root dependencies (Truffle, OpenZeppelin)

Contracts folder :

PredictionMarket.sol file

Code :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;
import "../node_modules/@openzeppelin/contracts/security/ReentrancyGuard.sol";
contract PredictionMarket is ReentrancyGuard {
    struct BetInfo {
        uint256 amount;
        uint8 choice; // 1 = Yes, 2 = No
        bool claimed;
    }

    struct Prediction {
        string question;
        uint256 deadline;
        address[] participants;
        mapping(address => BetInfo) bets;
    }
```

```

    mapping(address => bool) hasParticipated;
    uint256 totalYesBets;
    uint256 totalNoBets;
    uint256 outcome; // 0 = unresolved, 1 = Yes, 2 = No
    bool resolved;
}

uint256 public predictionCount;
mapping(uint256 => Prediction) public predictions;
address public owner;

event PredictionCreated(uint256 indexed predictionId, string question, uint256 deadline);
event BetPlaced(uint256 indexed predictionId, address participant, uint256 amount, uint8 choice);
event PredictionResolved(uint256 indexed predictionId, uint256 outcome);
event RewardClaimed(uint256 indexed predictionId, address claimant, uint256 amount);

modifier onlyOwner() {
    require(msg.sender == owner, "Only owner can call this.");
    _;
}

constructor() {
    owner = msg.sender;
}

function createPrediction(string memory _question, uint256 _deadline) external {
    require(_deadline > block.timestamp, "Deadline must be in the future.");
    uint256 predictionId = predictionCount++;
    Prediction storage p = predictions[predictionId];
    p.question = _question;
    p.deadline = _deadline;
    emit PredictionCreated(predictionId, _question, _deadline);
}

function placeBet(uint256 _predictionId, uint8 _choice) external payable nonReentrant {
    require(_choice == 1 || _choice == 2, "Invalid choice (1=Yes, 2=No).");
    Prediction storage p = predictions[_predictionId];
    require(block.timestamp < p.deadline, "Betting closed.");
    require(msg.value > 0, "Bet amount must be > 0.");

    BetInfo storage bet = p.bets[msg.sender];
    bet.amount += msg.value;
    bet.choice = _choice;

    if (!p.hasParticipated[msg.sender]) {
        p.participants.push(msg.sender);
        p.hasParticipated[msg.sender] = true;
    }

    if (_choice == 1) p.totalYesBets += msg.value;
    else p.totalNoBets += msg.value;

    emit BetPlaced(_predictionId, msg.sender, msg.value, _choice);
}

```

```

function resolvePrediction(uint256 _predictionId, uint256 _outcome) external onlyOwner {
    require(_outcome == 1 || _outcome == 2, "Invalid outcome.");
    Prediction storage p = predictions[_predictionId];
    require(!p.resolved, "Already resolved.");
    require(block.timestamp >= p.deadline, "Deadline not met.");
    p.outcome = _outcome;
    p.resolved = true;
    emit PredictionResolved(_predictionId, _outcome);
}

function claimReward(uint256 _predictionId) external nonReentrant {
    Prediction storage p = predictions[_predictionId];
    require(p.resolved, "Not resolved.");
    BetInfo storage bet = p.bets[msg.sender];
    require(bet.amount > 0 && !bet.claimed, "Invalid claim.");
    require(bet.choice == p.outcome, "You lost.");

    uint256 totalPool = (p.outcome == 1) ? p.totalYesBets : p.totalNoBets;
    uint256 reward = (bet.amount * (p.totalYesBets + p.totalNoBets)) / totalPool;

    bet.claimed = true;
    payable(msg.sender).transfer(reward);
    emit RewardClaimed(_predictionId, msg.sender, reward);
}

receive() external payable {}
}

```

Migrations folder :

1_deploy_contracts.js file

Code :

```

const PredictionMarket = artifacts.require("PredictionMarket");

module.exports = function (deployer) {
    deployer.deploy(PredictionMarket);
};

```

Note : there is no 2_deploy_mock_oracle.js # (Optional) Oracle integration
Please add it if possible

Test folder :

predictionMarket.test.js file :

```
const { expect } = require("chai");
const { BN, expectEvent, expectRevert } = require("@openzeppelin/test-helpers");
const PredictionMarket = artifacts.require("PredictionMarket");

contract("PredictionMarket", (accounts) => {
  let predictionMarket;
  const owner = accounts[0];
  const user1 = accounts[1];
  const user2 = accounts[2];

  beforeEach(async () => {
    predictionMarket = await PredictionMarket.new({ from: owner });
  });

  // Test 1: Contract deployment
  it("should deploy and set the owner", async () => {
    const contractOwner = await predictionMarket.owner();
    expect(contractOwner).to.equal(owner);
  });

  // Test 2: Create a prediction
  it("should create a new prediction", async () => {
    const question = "Will ETH hit $3000 by 2024?";
    const deadline = Math.floor(Date.now() / 1000) + 3600; // 1 hour from now

    const tx = await predictionMarket.createPrediction(question, deadline, { from: owner });

    // Check event emission
    expectEvent(tx, "PredictionCreated", {
      predictionId: new BN(0),
      question: question,
      deadline: new BN(deadline),
    });

    // Check stored prediction
    const prediction = await predictionMarket.predictions(0);
    expect(prediction.question).to.equal(question);
    expect(prediction.deadline).to.be.bignumber.equal(new BN(deadline));
  });

  // Test 3: Place a bet
  it("should allow users to place bets", async () => {
    const question = "Test Question";
    const deadline = Math.floor(Date.now() / 1000) + 3600;
    await predictionMarket.createPrediction(question, deadline, { from: owner });

    const betAmount = web3.utils.toWei("1", "ether");
    const choice = 1; // Yes

    await predictionMarket.placeBet(0, choice, {
      from: user1,
    });
  });
});
```

```

    value: betAmount,
  });

  const betInfo = await predictionMarket.predictions(0);
  expect(betInfo.totalYesBets).to.be.bignumber.equal(new BN(betAmount));
});

// Test 4: Resolve a prediction
it("should resolve a prediction and emit an event", async () => {
  const question = "Test Question";
  const deadline = Math.floor(Date.now() / 1000) - 3600; // Expired deadline
  await predictionMarket.createPrediction(question, deadline, { from: owner });

  const outcome = 1; // Yes
  const tx = await predictionMarket.resolvePrediction(0, outcome, { from: owner });

  // Check event and resolution status
  expectEvent(tx, "PredictionResolved", {
    predictionId: new BN(0),
    outcome: new BN(outcome),
  });
  const prediction = await predictionMarket.predictions(0);
  expect(prediction.resolved).to.be.true;
});

// Test 5: Claim rewards
it("should allow users to claim rewards", async () => {
  // Create and resolve a prediction
  const question = "Test Question";
  const deadline = Math.floor(Date.now() / 1000) - 3600;
  await predictionMarket.createPrediction(question, deadline, { from: owner });
  await predictionMarket.resolvePrediction(0, 1, { from: owner });

  // Place a winning bet
  const betAmount = web3.utils.toWei("1", "ether");
  await predictionMarket.placeBet(0, 1, {
    from: user1,
    value: betAmount,
  });

  // Claim reward
  const tx = await predictionMarket.claimReward(0, { from: user1 });
  expectEvent(tx, "RewardClaimed", {
    predictionId: new BN(0),
    claimant: user1,
  });
});

```

Frontend folder :

Public folder :

index.html file:

code :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-
manifest/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
      Notice the use of %PUBLIC_URL% in the tags above.
      It will be replaced with the URL of the `public` folder during the build.
      Only files inside the `public` folder can be referenced from the HTML.

      Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
      work correctly both with client-side routing and a non-root public URL.
      Learn how to configure a non-root public URL by running `npm run build`.
    -->
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
    <!--
      This HTML file is a template.
      If you open it directly in the browser, you will see an empty page.

      You can add webfonts, meta tags, or analytics to this file.
      The build step will place the bundled scripts into the <body> tag.

      To begin the development, run `npm start` or `yarn start`.
      To create a production bundle, use `npm run build` or `yarn build`.
    -->
  </body>
</html>
```

Src folder :

Components folder :

CreatePrediction.js file :

Code :

```
import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";

const CreatePrediction = ({ contract, account }) => {
  const [question, setQuestion] = useState("");
  const [deadline, setDeadline] = useState("");

  const handleCreate = async () => {
    if (!contract || !account) return;
    const deadlineTimestamp = Math.floor(new Date(deadline).getTime() / 1000);
    await contract.methods.createPrediction(question, deadlineTimestamp).send({ from: account });
    alert("Prediction created successfully!");
  };

  return (
    <div className="mb-5">
      <h2>Create Prediction</h2>
      <Form>
        <Form.Group className="mb-3">
          <Form.Label>Question</Form.Label>
          <Form.Control
            type="text"
            placeholder="Enter prediction question"
            value={question}
            onChange={(e) => setQuestion(e.target.value)}
          />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Deadline</Form.Label>
          <Form.Control
            type="datetime-local"
            value={deadline}
            onChange={(e) => setDeadline(e.target.value)}
          />
        </Form.Group>
        <Button variant="primary" onClick={handleCreate}>
          Create Prediction
        </Button>
      </Form>
    </div>
  );
};

export default CreatePrediction;
```

PredictionList.js file :

Code :

```
import React from "react";
import { Card, Button, ListGroup } from "react-bootstrap";
import BetForm from "./BetForm";

const PredictionList = ({ predictions, contract, account }) => {
  return (
    <div>
      <h2>Active Predictions</h2>
      {predictions.map((prediction) => (
        <Card key={prediction.id} className="mb-3">
          <Card.Body>
            <Card.Title>{prediction.question}</Card.Title>
            <Card.Text>Deadline: {prediction.deadline}</Card.Text>
            <BetForm prediction={prediction} contract={contract} account={account} />
          </Card.Body>
        </Card>
      ))}
    </div>
  );
};

export default PredictionList;
```

BetForm.js file :

Code :

```
import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";

const BetForm = ({ prediction, contract, account }) => {
  const [amount, setAmount] = useState("");
  const [choice, setChoice] = useState(1);

  const handleBet = async () => {
    if (!contract || !account) return;
    await contract.methods.placeBet(prediction.id, choice).send({
      from: account,
      value: web3.utils.toWei(amount, "ether"),
    });
    alert("Bet placed successfully!");
  };

  return (
    <Form>
      <Form.Group className="mb-3">
        <Form.Label>Amount (ETH)</Form.Label>
        <Form.Control
          type="number"
          placeholder="Enter amount"
          value={amount}
        />
      </Form.Group>
    </Form>
  );
};
```

```

        onChange={(e) => setAmount(e.target.value)}
      />
    </Form.Group>
    <Form.Group className="mb-3">
      <Form.Label>Choice</Form.Label>
      <Form.Select value={choice} onChange={(e) => setChoice(parseInt(e.target.value))}>
        <option value={1}>Yes</option>
        <option value={2}>No</option>
      </Form.Select>
    </Form.Group>
    <Button variant="success" onClick={handleBet}>
      Place Bet
    </Button>
  </Form>
);
};

export default BetForm;

```

src folder : (out of components) :

App .js file :

Code :

```

import React, { useEffect, useState } from "react";
import Web3 from "web3";
import { CONTRACT_ADDRESS, CONTRACT_ABI } from "../config";
import CreatePrediction from "../components/CreatePrediction";
import PredictionList from "../components/PredictionList";
import "bootstrap/dist/css/bootstrap.min.css";
import "../styles.css";

const App = () => {
  const [web3, setWeb3] = useState(null);
  const [account, setAccount] = useState(null);
  const [contract, setContract] = useState(null);
  const [predictions, setPredictions] = useState([]);

  useEffect(() => {
    const init = async () => {
      if (window.ethereum) {
        const web3Instance = new Web3(window.ethereum);
        setWeb3(web3Instance);
        await window.ethereum.request({ method: "eth_requestAccounts" });
        const accounts = await web3Instance.eth.getAccounts();
        setAccount(accounts[0]);

        const contractInstance = new web3Instance.eth.Contract(CONTRACT_ABI,
          CONTRACT_ADDRESS);
        setContract(contractInstance);
      }
    };
    init();
  }, []);

  return (
    <div>
      <CreatePrediction />
      <PredictionList />
    </div>
  );
};

export default App;

```

```

    // Load predictions
    const predictionCount = await contractInstance.methods.predictionCount().call();
    const predictions = [];
    for (let i = 0; i < predictionCount; i++) {
        const prediction = await contractInstance.methods.predictions(i).call();
        predictions.push({
            id: i,
            question: prediction.question,
            deadline: new Date(prediction.deadline * 1000).toLocaleString(),
            resolved: prediction.resolved,
            outcome: prediction.outcome,
        });
    }
    setPredictions(predictions);
}
};
init();
}, []);

return (
    <div className="container mt-5">
        <h1 className="text-center mb-4">Prediction Market</h1>
        <p className="text-center">Connected Account: {account}</p>

        <CreatePrediction contract={contract} account={account} />
        <PredictionList predictions={predictions} contract={contract} account={account} />
    </div>
);
};

export default App;

```

config.js file :

code :

```

export const CONTRACT_ADDRESS = "0x..."; // Replace with your contract address
export const CONTRACT_ABI = [
    {
        "inputs": [],
        "stateMutability": "nonpayable",
        "type": "constructor"
    },
    {
        "anonymous": false,
        "inputs": [
            {"indexed": true, "internalType": "uint256", "name": "predictionId", "type": "uint256"},
            {"indexed": false, "internalType": "address", "name": "participant", "type": "address"},
            {"indexed": false, "internalType": "uint256", "name": "amount", "type": "uint256"},
            {"indexed": false, "internalType": "uint8", "name": "choice", "type": "uint8"}
        ],
        "name": "BetPlaced",
        "type": "event"
    },
    // ... (include all other ABI entries from the JSON file)
]

```

```

    "inputs": [
      {"internalType": "uint256", "name": "_predictionId", "type": "uint256"}
    ],
    "name": "claimReward",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "stateMutability": "payable",
    "type": "receive"
  }
];

```

index.js file :

code :

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();

```

styles.css file :

code :

```

body {
  font-family: Arial, sans-serif;
}

.container {
  max-width: 800px;
  margin: 0 auto;
}

.card {
  margin-bottom: 20px;
}

```

Package.json file (frontend dependency)

```

{
  "name": "frontend",

```

```

"version": "0.1.0",
"private": true,
"dependencies": {
  "bootstrap": "^5.3.3",
  "cra-template": "1.2.0",
  "react": "^19.0.0",
  "react-bootstrap": "^2.10.9",
  "react-dom": "^19.0.0",
  "react-scripts": "^3.0.1"
},
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
  ]
},
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
}
}

```

Package.json file (root dependency) :

```

{
  "dependencies": {
    "@openzeppelin/contracts": "^4.9.6",
    "@openzeppelin/test-helpers": "^0.5.16",
    "chai": "^5.1.2",
    "truffle": "^5.5.7",
    "web-vitals": "^4.2.4",
    "web3": "^4.16.0"
  },
  "name": "miniproject",
  "version": "1.0.0",
  "main": "script.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
}

```

```
"author": "",  
"license": "ISC",  
"description": ""  
}
```

truffle-config.js file :

```
module.exports = {  
  contracts_build_directory: "./build",  
  // Add paths configuration  
  paths: {  
    sources: "./contracts",  
    tests: "./test",  
    cache: "./cache",  
    artifacts: "./build/contracts"  
  },  
  networks: {  
    development: {  
      host: "127.0.0.1",  
      port: 7545,  
      network_id: "*",  
    },  
  },  
  compilers: {  
    solc: {  
      version: "0.8.28",  
    },  
  },  
};
```

